Integrated Inventory Donation Tracking System for Non-Profits

Software Requirements Specification (SRS) and
Project Development Plan (PDP)

Insiyah Ujjainwala
Taaliah Ayub
Haley Johnson
Andrew Goetz
Nika Khajehpour
Victor Moucattash
Alyssa Wang

Version 0

November 5, 2025

McMaster University
Department of Computing and Software
2025–2026

Contents

1 Project Title **3**

2 Version **3**

3 Team Members & Their Roles **3**

4 Abbreviations, Notations, and Definitions **3**

5 Table of Contents and Contributions **3**

6 Purpose of the Project **3**

7 Client and Stakeholders **4**

8 Project Constraints **4**

9 Functional Requirements **4**
9.1 P0 - MVP (Minimum Viable Product) . . . 4
9.2 P1 - Next Feature Set . . . 5
9.3 P2 - Stretch Features (Non-Critical) . . . 6
9.4 P3 - Future Features (Nice to Have) . . . 8

10 Data and Metrics **9**
10.1 Data Sources and Use . . . 9
10.2 Dataset Links or Simulation Plan . . . 9
10.3 Performance Metrics and Goals . . . 10

11 Non-Functional Requirements **10**
11.1 Look and Feel Requirements . . . 10
11.2 Usability and Humanity Requirements . . . 10
11.3 Performance and Speed Requirements . . . 10
11.4 Security and Privacy . . . 10
11.5 Legal Considerations . . . 10

12 Predicted Risks and Issues **11**

13 Team Meeting and Communication Plan **12**

14 Team Member Roles **12**
14.1 Functional and Non-Functional Assignments . . . 12
14.2 Team Coordinator and their Responsibilities . . . 13

15 Workflow Plan **13**
15.1 Git/GitHub Usage and Branching Strategy . . . 13
15.2 Agile or Iterative Development Approach . . . 14
15.3 Data Storage . . . 14
15.4 Compute Heavy Tasks . . . 14
15.5 Tools and Methods to Achieve Requirements . . . 14

16 Proof of Concept Demonstration Plan **15**

17 Technology Stack **16**

17.1 Programming Languages and Frameworks 16

17.2 ML Libraries (If Applicable) 16

17.3 Hardware and Other Technological Considerations 16

18 Project Schedule: Gantt Chart **17**

1 Project Title

Integrated Inventory Donation Tracking System for Non-Profits

2 Version

Version 0

3 Team Members & Their Roles

Name	Email	Role
Insiyah Ujjainwala	ujjainwi@mcmaster.ca	Frontend Developer, Data Analyst
Taaliah Ayub	ayubt@mcmaster.ca	Machine Learning Developer
Haley Johnson	johnsh20@mcmaster.ca	Full-Stack Developer
Andrew Goetz	goetza3@mcmaster.ca	Machine Learning Developer
Nika Khajehpour	khajehn@mcmaster.ca	Project Manager, Frontend Developer
Victor Moucattash	moucattv@mcmaster.ca	Machine Learning Developer
Alyssa Wang	wanga121@mcmaster.ca	Full-Stack Developer

4 Abbreviations, Notations, and Definitions

NP: Non-Profit **ML:** Machine Learning **DF:** Demand Forecasting

5 Table of Contents and Contributions

The following table outlines the main sections of this document and the team members primarily responsible for their completion.

Section	Contributors
Purpose of the Project	Taaliah Ayub
Client and Stakeholders	Victor Moucattash, Nika Khajehpour
Project Constraints	Alyssa Wang, Insiyah Ujjainwala
Functional Requirements	Haley Johnson
Data and Metrics	Andrew Goetz
Non-Functional Requirements	Victor Moucattash, Nika Khajehpour
Risks and Issues Predicted	Alyssa Wang, Insiyah Ujjainwala
Team Meeting and Communication Plan	Victor Moucattash, Nika Khajehpour
Team Member Roles and Coordinator Responsibilities	All Members
Workflow Plan	Haley Johnson, Andrew Goetz
Proof of Concept Demonstration Plan	Andrew Goetz
Technology Stack	Insiyah Ujjainwala, Alyssa Wang
Project Scheduling	All Members

6 Purpose of the Project

A vital part of running NP organizations dedicated to serving the community and the disadvantaged is keeping track of current inventory and what resources are in demand. Currently, these organizations often rely on manual record-keeping with pen and paper, basic tools like Excel, or costly software solutions. The aim of this project is to address these aspects of managing NPs. Features such as inventory management using barcode scanning and summary analytics via a dashboard interface will assist with organization of available supplies. ML concepts will be applied in order to forecast demand of current or desired supplies, and estimate expenses of the predicted demand. In developing this software, the hope is to provide a convenient and free alternative to currently available inventory tracking programs, and, as a result, help cut down on time and expenses being used on paid or inconvenient solutions that could be otherwise put back into philanthropic pursuits.

Software Requirements Specification

7 Client and Stakeholders

Client

Our main client is small to mid-sized NP organizations, especially local food banks. Right now, many of them use spreadsheets or paid software that's either inefficient or too expensive. Our system is being built to give them a free, simple, and sustainable tool for tracking inventory and donations.

Stakeholders

- **Staff and volunteers**: They'll be the main users, entering items, scanning barcodes, and checking inventory. Since volunteers often rotate, the system has to be straightforward and require little training.
- **Donors**: While not direct users, they benefit from more transparency and knowing their contributions are being tracked and reported properly.
- **Auditors and regulators**: They need accurate, reliable reports for compliance and accountability. Automated reporting makes this easier.
- **NP management**: They'll use dashboards and forecasting features to plan ahead, allocate resources, and make better decisions.
- **Community members**: Even though they won't touch the system, they're impacted as the end recipients. A smoother process means less waste, faster distribution, and better access to essentials.

In short, the client is NPs, but the impact extends to everyone connected to them: the people running day-to-day operations, the people donating, and the people relying on their services.

8 Project Constraints

- **Web-Based Platform**: The system is being developed as a web-application to ensure broad accessibility across devices and minimize setup requirements for NPs. A mobile-responsive web interface will be implemented for barcode/item scanning functionality rather than native mobile applications (though a native mobile app/lightweight native wrapper remains a P2 stretch goal).
- **Client Scope**: The project is explicitly scoped for small to mid-sized NPs, with local food banks as the primary archetype. This focus constrains the feature set and complexity to problems faced by organizations with limited budgets and technical staff. Support for large, multi-branch organizations is explicitly defined as a P3 feature.
- **Open-Source and Cost-Effective**: The project will be developed and released as a free and open-source solution (under the MIT License), using a cost-effective, open-source technology stack. The system must be easy for NPs to set up and require minimal ongoing upkeep.
- **Feature Prioritization**: Payment processing for monetary donations (P3) will rely on third-party integration (e.g., Zeffy, PayPal) rather than custom payment infrastructure due to security, compliance, and resource constraints.

9 Functional Requirements

The following functional requirements are based on our current vision of the finished product and outline the features we aim to deliver. We anticipate that certain aspects - including front-end and back-end functionality, data sources, and the number of P2 features implemented - may evolve as development progresses and technical constraints become clearer.

9.1 P0 - MVP (Minimum Viable Product)

---

1. Inventory Management

Goal: Allow users to add, update, remove, and track items in inventory efficiently.

Frontend Features:

- **Barcode Scanning UI:** Uses the device camera to scan barcodes and autofill item data if recognized; supports manual entry when not found.
- **Item Management Interface:** Provides a form for adding or editing items (name, category, quantity,

expiration date) and a searchable, filterable list view.
- **Location Selector:** Includes a dropdown menu for assigning storage locations when adding or editing items.
- **Bulk Upload:** Allows CSV or Excel uploads for logging multiple items simultaneously, with options to adjust quantities before submission.

Backend Features

- **Barcode Lookup & Registration:** Includes endpoints to search for existing barcodes and register new barcode mappings to item names.
- **Inventory Schema and Upload:** Defines database fields such as `item_id, name, category, quantity, expiration_date, location_id, date_added, last_modified.`
- **Bulk Import Handler:** Validates and parses CSV or Excel uploads, performing batch inserts or updates.
- **Logging Mechanism:** Stores timestamped entries for each add, update, or remove action to ensure full traceability.

Data Needed

- Item metadata: `item_id, name, category, barcode, quantity, unit, expiration_date, location_id, date_added`
- User performing each transaction

---

2. Dashboard

Goal: Provide summary analytics and quick insights into inventory.

Frontend Features

- **Graphical Reports and Widgets:** Display total inventory by category, low-stock alerts, and expiring-item warnings through interactive visual components.
- **Export Options:** Generate downloadable reports in PDF or CSV format for record-keeping and analysis.

Backend Features

- **Aggregate Queries:** Compute inventory totals, low-stock counts, and near-expiry items using optimized database queries.
- **Scheduled Jobs:** Automatically flag items approaching expiry within defined thresholds (e.g., 30 days) to support timely restocking.

Data Needed

- Item metadata stored within the inventory database.

---

3. Documentation

- **Setup Guide:** Includes installation instructions, environment configuration, and deployment steps.
- **User Manual:** Provides a non-technical guide explaining how to use the system for inventory tracking and reporting.
- **Unit Testing:** All team members contribute to testing as modules are developed.

9.2 P1 - Next Feature Set

---

1. Demand Forecasting

Goal: Predict demand for items based on historical data and patterns.

Frontend Features

- **Forecast Dashboard:** Displays predicted item shortages and demand trends.
- **Visualization Charts:** Shows projected future demand (e.g., weekly or monthly) using graphical insights.

Backend Features

- **Machine Learning Model:** Uses historical inflow and outflow data to generate demand predictions.
- **Data Pipeline:** Aggregates and processes past transactions to feed into the forecasting model.

Data Needed

- **FreshRetailNet-50K:** A demand dataset including perishable items, available at `https://arxiv.org/abs/2505.16319` and `https://huggingface.co/datasets/Dingdong-Inc/FreshRetailNet-50K`.

1.5. Procurement Advising / Wish Lists (Part of DF)

Goal: Help organizations prioritize what to restock or request from donors.

Frontend Features

- **Wish List Form:** Allows organizations to manually create and manage lists of desired items.

Backend Features

- **Stock vs Demand Algorithm:** Compares current inventory with forecasted demand and integrates results from the demand forecasting model.

---

2. Role-Based Access / Authentication

Goal: Ensure proper access control and data security.

Frontend Features

- **Login and Registration:** Provides user authentication through secure forms.
- **Role-Based Interface:** Displays different UI elements and permissions for volunteers, staff, and admins.

Backend Features

- **Authentication Endpoints:** Implements JWT or session-based authentication.
- **Roles Table:** `(role_id, name, permissions)`.
- **User Table:** `(user_id, name, email, role_id)`.

Data Needed

- User credentials, role assignments, and role-permission mappings.

9.3 P2 - Stretch Features (Non-Critical)

---

1. Audit

Goal: Extend the existing Dashboard to include audit and reporting features that allow nonprofits to track their yearly performance, including donations, fundraisers, and key inventory activities.

Frontend Features

- **Donation and Fundraiser Tracking:** Enables users to log donation details (amount, source, date) and record fundraiser events with goals and total funds raised.
- **Audit Dashboard Extension:** Expands the main dashboard to display yearly summaries of donations, fundraisers, and inventory activities.
- **Performance Reports:** Generates downloadable year-end performance documents highlighting key metrics such as total donations and most requested items.
- **Filtering and Visualization:** Allows filtering by date range, campaign, or location, with visual charts for donation trends and event outcomes.

Backend Features

- **Data Aggregation:** Collects and summarizes data from donations, fundraisers, and inventory transactions throughout the year.
- **Audit Logging:** Records major financial and inventory-related actions to maintain transparency and accuracy in annual reporting.
- **Report Generation:** Compiles yearly summaries into downloadable PDF or CSV files for compliance and stakeholder reporting.
- **Donations & Fundraisers Schema and Upload:** Defines database tables and fields for structured capture and reporting.
  `donations` — `donation_id, amount, date_received, source, donor_name`.
  `fundraisers` — `fundraiser_id, name, goal_amount, total_raised`.

Data Needed

- Donation and fundraiser data (amount, date, source, campaign), inventory transactions, and user activity logs.
- Inventory and user activity data required for annual performance summaries.

---

2. Load Balancing

Goal: Maintain consistent performance and reliability under concurrent usage.

Backend Features

- **Application-Level Load Balancing:** Distributes requests evenly across multiple instances to optimize performance.
- **Cloud Integration:** Can leverage cloud-based services for scaling when multiple organizations are active simultaneously.

---

3. Computer Vision (Non-Barcoded Items)

Goal: Recognize non-barcoded items, such as clothing or fresh produce, using image-based classification.

Frontend Features

- **Camera Integration:** Uses the device camera to capture item images and trigger recognition.
- **Manual Override:** Provides an option for users to manually enter item details if the model's prediction is uncertain.

Backend Features

- **ML Model:** Implements a trained machine learning model to identify non-barcoded items (e.g., clothing or produce) and suggest item categories.
- **Image Processing Pipeline:** Handles image upload, resizing, and preprocessing before inference to ensure consistent input quality.
- **Prediction API:** Exposes an endpoint for the frontend to send images and receive classification results.

Data Needed

We plan to use publicly available image datasets containing clothing items (such as shirts, pants, and jackets) and fresh produce (like fruits and vegetables) to train our computer vision model. Since this is a stretch feature, we haven't finalized the exact datasets yet, but the goal is to enable the model to recognize non-barcoded donations and suggest likely item categories through the camera interface.

---

4. Mobile App (For Scanning Barcodes)

Goal: Extend barcode scanning functionality to mobile devices for faster and more accessible inventory entry.

Frontend Features

- **Mobile Camera Access:** Provides scanning functionality directly through the mobile app interface.

Backend Features

- **Integration:** Connects with existing inventory management backend endpoints for seamless data synchronization.

Data Needed

- `item_id, name, category, barcode, quantity, unit, expiration_date, location_id, date_added`, and user performing each transaction.

9.4 P3 - Future Features (Nice to Have)

---

1. Multiple Locations

Goal: Support organizations with multiple warehouses or branches.

Frontend Features

- **Location Selector:** Dropdown or map-based interface allowing users to switch between locations.
- **Location-Specific Dashboards:** Visualize stock levels, item movements, and performance metrics per branch.
- **Transfer Interface:** Form-based tool to initiate and confirm transfers between locations.

Backend Features

- **Location Management API:** Handles CRUD (Create, Read, Update, and Delete) operations for locations and maintains relationships with inventory data.
- **Inventory Segmentation:** Enforces data separation per `location_id` to avoid data overlap.
- **Transfer Logging:** Records inter-location transfers with details like item, quantity, timestamps, and user IDs.

Data Needed

- `location_id, organization_id`: Unique identifiers linking inventory items to specific branches.
- `transfer_id, source_location, destination_location, item_id, quantity, timestamp, user_id`.
- Aggregated data for location-based dashboards (e.g., total stock, recent transfers, low-stock alerts). Collaboration with built-out dashboard feature.

---

2. Monetary Donations

Goal: Integrate with donation platforms to track financial contributions and sources.

Frontend Features

- **Donation Summary Dashboard:** Displays total donations, recent contributions, and funding trends.
- **Donation History View:** Enables filtering by donor, amount, or date range.
- **Platform Connection UI:** Allows authorized users to connect or disconnect external donation services (e.g., Zeffy API).

Backend Features

- **Donation Platform Integration:** Connects with external APIs (e.g., Zeffy) to retrieve and log donation data.
- **Donation Tracking:** Records donation sources, amounts, and timestamps for financial reporting.
- **Automated Sync:** Periodically updates donation records from external services.

Data Needed

- `donation_id, donor_id, amount, source, platform, timestamp, transaction_status`.
- `organization_id` for linking donations to a specific organization.
- Aggregated financial data for dashboard visualizations (e.g., monthly totals, source breakdowns).

---

3. Relationship Management

Goal: Develop the tool into an all-in-one non-profit management platform by implementing a built-in Customer Relationship Management (CRM) system to organize and maintain relationships with donors, partners, and sponsors.

Frontend Features

- **Contact Dashboard:** Displays a searchable list of donors and partners with contact details, organization affiliation, and recent activity.
- **Profile View:** Shows detailed contact information, donation history, communication records, and tags (e.g., "recurring donor," "corporate partner," "volunteer sponsor").
- **Interaction Logging:** Provides a simple form to record calls, emails, or meetings with each contact.
- **Email/Outreach Tracking (Stretch Goal):** Potential integration with email automation tools (e.g., Mailchimp API or SMTP-based systems) to monitor outreach campaigns and responses.

Backend Features

- **CRM Schema:** `contact_id, name, email, organization, type (donor/partner), total_donations, last_donation_date, notes, tags`.
- **Communication Logs Table:** `log_id, contact_id, date, type (email/meeting/call), notes, user_id`.
- **API Endpoints:** Provides CRUD operations for contacts and communication logs (e.g., `/contacts`, `/contacts/:id/logs`, `/contacts/search`).
- **Analytics Module:** Aggregates donation totals by donor, tracks engagement over time, and flags inactive donors for follow-up.

Data Needed

- Contact information: name, organization, email, phone, tags.
- Donation data: `donation_id, donor_id, amount, date, type`.
- Communication records: type, date, notes, and staff user who logged the interaction.

---

10 Data and Metrics

10.1 Data Sources and Use

We will be building a DF model to predict the needs of food banks/shelters. The model takes into account perishable items. The main features/predictors that will be used are:

- **Historical Demand**: Past consumption for each item over a finite period
- **Stock indicator**: The current level of stock/inventory of particular items
- **Seasonal Demand**: Certain times of year increase demand for certain items
- **Holiday Spikes**: Some Holidays spike demand for very specific items
- **Weather**: Temperature, Precipitation, etc, correlated to seasonal demand
- **Trends**: increasing demand for some items over time

10.2 Dataset Links or Simulation Plan

FreshRetailNet-50K:

https://arxiv.org/abs/2505.16319 https://huggingface.co/datasets/Dingdong-Inc/FreshRetailNet-50K

This is a Demand Dataset for Latent Demand Recovery and Forecasting in Fresh Retail. We will start by training and experimenting using the FreshRetailNet-50K dataset. We will reinterpret sales as consumption, and standardize/remove features for price/discount (unless they can be repurposed somehow). The model will learn seasonality through the date/time features and holiday features.

10.3 Performance Metrics and Goals

For the DF model, we will evaluate its performance using error-based metrics like forecast bias (for systematic over-/under forecasting), MAE (mean absolute error), and WAPE (Weighted Absolute Percentage Error), a good forecasting metric that calculates the total absolute forecast error as a percentage of the total actual demand.

Our goal is to achieve:

- <30% WAPE (a bar for reasonable performance)
- a bias within 10% of actual demand

These metrics are relevant for our DF system because WAPE is specifically a metric for forecasting, which accounts for the absolute error normalized by the total demand. While WAPE tells us how big the error is, Forecasting Bias tells us which direction this error is in.

11 Non-Functional Requirements

11.1 Look and Feel Requirements

The interface will be simple, clean, and consistent. NPs often rely on staff and volunteers who don't have much technical background, so the design needs to be approachable and not overwhelming. Clear icons, plain language labels, and a straightforward dashboard are priorities. We will also avoid clutter by keeping screens focused on one main task at a time (e.g., scanning, reporting, donation tracking). Text and labels will be clear and legible, and should be readable for colour blind users.

Immediate feedback on scans (beep, vibration, or a green check). Show a confirmation card with editable fields (name, quantity, expiry, location) before committing.

Use color cues:

- Green for success
- Amber for "check this" (low confidence match)
- Red for errors

11.2 Usability and Humanity Requirements

The system should be easy to learn and forgiving to mistakes. For example, if a barcode scan or manual entry is wrong, users should be able to undo or correct it quickly without breaking the workflow. Because volunteers often rotate, the platform should require little to no formal training; ideally, someone should be able to figure out the basics on their first use. Accessibility is also important: we'll aim to follow basic accessibility guidelines (readable fonts, good contrast, simple navigation) so the tool works for a wide range of users.

11.3 Performance and Speed Requirements

The system should be lightweight and responsive, since many NPs may not have access to high-end hardware or fast internet. Inventory updates, barcode scans, and dashboard views should all load within a few seconds at most. To make this possible, we'll design with scalability in mind; for example, the platform will incorporate load balancing techniques to support a handful of users working at the same time without slowing down noticeably.

11.4 Security and Privacy

Because the system deals with donation records, user accounts, and potentially sensitive organizational data, we need to ensure strong protection. This includes role-based access so only authorized users can make changes, encrypted storage for important information, and secure login (e.g., JWT authentication). We also want to minimize the amount of personal data collected (only what's truly needed to run the platform) to reduce risks and build trust with NPs.

11.5 Legal Considerations

Our system must follow privacy and data protection regulations (for example, PIPEDA in Canada, and GDPR if the tool expands internationally). Since we're making this an open-source project, we will use the MIT License, which allows others to freely use, copy, and adapt our code, while protecting us from liability and requiring attribution. This makes it ideal for NPs who may want to continue using or modifying the system after our capstone ends. If we later add donation processing, we'll also make sure to comply with the

financial and legal standards of whichever payment platform we integrate (e.g., PayPal, Zeffy).

12 Predicted Risks and Issues

While the project aims to deliver a reliable and impactful inventory management system, several technical and operational risks have been identified that could affect scope, performance, or adoption.

- **Scalability and load balancing**: There's a risk we may not be able to fully implement load balancing. Without it, the backend might struggle with concurrent requests if multiple users are active at the same time.
- **Monetary donations**: Adding support for donations without tying into a third-party processor (like PayPal or Zeffy) could be difficult. If we can't resolve this, the donation feature may be limited or dependent on external services, which reduces flexibility for NPs.
- **User feedback and adoption**: The usefulness of the tool depends on feedback from actual NPs. If we can't connect with organizations for testing and input, the system may not align well with their real-world needs.
- **ML accuracy**: DF relies on having enough quality training data. If the dataset is too limited or inconsistent, predictions may be inaccurate and could lead to poor resource planning. Also we predict potential issues with the integration of our frontend/backend and ML systems.
- **Security and data leakage**: Since the system will store organizational data and user accounts, there's always a risk of unauthorized access or data leaks. Weak authentication, poor encryption, or unsafe handling of sensitive information could expose NPs to risks and harm trust in the platform.

Project Development Plan

13 Team Meeting and Communication Plan

We'll be using a mix of tools to stay organized and keep communication easy:

- **Google Workspace**: for collaborative work on documents, slides, and notes. This lets everyone edit in real time and ensures we're always working from the same version.
- **Microsoft Teams**: for meetings, quick updates, and document sharing. We'll use Teams for our weekly check-ins as well as extra discussions whenever something comes up that needs attention right away.
- **Messenger**: for quick or urgent matters, since messages there are often seen faster. This helps us resolve things quickly if someone isn't checking Teams.
- **Trello**: for task tracking. Each task will be added to Trello, and we'll use it to assign responsibilities, set deadlines, and keep track of progress.
- As a group, we've set a standing weekly meeting on Wednesdays at 6:30 pm. We'll use that time to review progress, raise blockers, and plan next steps. As the project moves forward, we'll schedule additional meetings during the week as needed to keep up with deadlines and workload.

This setup makes sure we have both structure (through Trello and Google Workspace) and flexibility (through Teams, Messenger, and extra meetings), so we can manage our work efficiently and keep everyone aligned.

14 Team Member Roles

14.1 Functional and Non-Functional Assignments

Priority	Module / Feature Area	Functional Requirements	Primary Developer(s)	Secondary Developer(s)
P0	Inventory	Frontend - Barcode Scanning UI	Haley, Nika	Insiyah, Taaliah
P0	Inventory	Frontend - Item Management Interface	Insiyah, Haley	Nika, Taaliah
P0	Inventory	Frontend - Location Selector	Taaliah	Haley
P0	Inventory	Frontend - Bulk Upload (CSV/Excel)	Insiyah	Taaliah
P0	Inventory	Backend - Inventory Schema and Upload	Haley	Insiyah
P0	Inventory	Backend - Barcode Lookup & Registration Endpoints	Nika	Taaliah
P0	Inventory	Backend - Bulk Import Handler	Taaliah	Insiyah
P0	Inventory	Backend - Logging Mechanism	Haley	
P0	Dashboard	Frontend - Graphical Reports & Widgets	Victor, Nika	Andrew
P0	Dashboard	Frontend - Export Options (PDF/CSV)	Andrew	Victor
P0	Dashboard	Backend - Aggregate Queries	Taaliah	Andrew
P0	Dashboard	Backend - Scheduled Jobs	Victor	Andrew
P0	Documentation	Setup Guide, User Manual	All	
P0	Documentation	Unit Testing	All	
P1	Demand Forecasting	Frontend - Forecast Dashboard & Visualization	Insiyah, Victor	Andrew
P1	Demand Forecasting	Backend Forecasting Model	Taaliah, Andrew, Victor	Insiyah
P1	Demand Forecasting	Data Pipeline for Historical Outflows	Andrew, Taaliah	Victor
P1	Procurement Advising	Procurement Advising / Wish Lists	Insiyah	Victor
P1	Authentication	Role-Based Access & Authentication	Alyssa, Andrew	
P2	Audit	Audit Dashboard and Visualization	Insiyah, Nika, Alyssa	
P2	Audit	Performance Reports	Insiyah, Nika	Haley
P2	Audit	Audit Data Aggregation	Insiyah, Haley	

Priority	Module / Feature Area	Functional Requirements	Primary Developer(s)	Secondary Developer(s)
P2	Audit	Audit Logging and Report Generation	Insiyah	
P2	Load Balancing	Application-Level Load Balancing	Taaliah, Haley	
P2	Load Balancing	Cloud Integration	Taaliah	Haley
P2	Computer Vision	Camera Integration (Frontend)	Nika	Insiyah
P2	Computer Vision	Manual Override	Insiyah	Nika
P2	Computer Vision	ML Model	Nika	Insiyah
P2	Computer Vision	Image Processing Pipeline	Nika	Insiyah
P2	Computer Vision	Prediction API	Insiyah	Nika
P2	Mobile App	Mobile Camera Access	Haley	Alyssa
P2	Mobile App	Backend Integration with Inventory API	Alyssa	Haley
P3	Multi-Location Support	Support for Storage in Multiple Locations	All	
P3	Monetary Donations	Monetary Donations Integration	Insiyah	
P3	Relationship Management	Relationship Management (CRM)	Victor	

14.2 Team Coordinator and their Responsibilities

The team's designated coordinator is **Nika Khajehpour**. The coordinator is responsible for overseeing project scheduling, leading meetings, and ensuring that tasks remain on track according to established milestones and deadlines. While the coordinator helps manage workflow and communication, they do not hold authority over individual development decisions, design choices, or technical architecture.

The coordinator has the authority to propose timelines, set deadlines, and recommend task redistribution when team members encounter blockers. However, any redistribution of work must be agreed upon by the rest of the development team. The coordinator does not have disciplinary authority and cannot remove team members from the group.

Key Responsibilities

- **Meeting Management:** Schedule and facilitate team meetings, ensuring discussions are productive and aligned with project objectives.
- **Progress Tracking:** Monitor task completion and milestone progress, following up with members as needed to maintain momentum.
- **Communication Liaison:** Serve as the primary point of contact between the team and course staff (instructors and TAs), sharing updates or clarifying requirements.
- **Task Coordination:** Assist in assigning and updating Trello tasks to reflect ongoing development work.
- **Record Keeping:** Document meeting notes, key decisions, and next steps for future reference and accountability.
- **Conflict Resolution:** Help resolve scheduling or workload conflicts to maintain smooth team collaboration.

In summary, the coordinator ensures the project progresses efficiently by managing schedules, facilitating communication, and maintaining alignment across all team members and stakeholders.

15 Workflow Plan

15.1 Git/GitHub Usage and Branching Strategy

We will have a protected main branch that requires PR approval from two team members before the ability to merge into it. When team members approve a PR, they should review the code to ensure that new features work as intended, and that the code follows best practices.

Branches will follow the naming convention `<fix/feat>/<description>`. The prefix `feat` is used for new features or functionality, while `fix` is used for bug fixes.

For example:

- `fix/authentication`
- `feat/add-user-login-redirect`

When working on a branch, team members will pull often to get the latest commits from Main. This will help to minimize merge conflicts.

Team members working on the same branch should coordinate often to know what features are being implemented and where, to reduce merge conflicts. We will try to keep separate branches per team member when possible.

We will not be using issues, and instead link branches and PRs to Trello tasks. PRs will reference Trello "card numbers", this creates a relationship between every PR and Trello task.

15.2 Agile or Iterative Development Approach

No, we will not be doing scrum and sprint planning.

We will however be using Trello (similar to Jira) to plan tasks. All development tasks will be assigned in Trello and we will meet frequently as a team to plan and update on progress. While we will follow the agile SDLC philosophy we will not stick to strict sprint schedules. We will have an iterative design and development process with sufficient testing as well as scheduling tasks and establishing deadlines.

15.3 Data Storage

For app data we will be using a relational database (PostgreSQL). This will be used for read and writing user data and implementing some business logic.

We will store ML datasets seperately. These will be in cloud object storage, maintaining raw, processed, and feature buckets. We will use a tool with versioning so experiments are reproducible. Tools like Google cloud Storage or Azure Storage are viable options.These tools provide encryption along with other valuable features, like integration with ML platforms, storing raw, processed and feature-engineered tables.

15.4 Compute Heavy Tasks

We will investigate using the Computing and Software departments servers for some compute heavy tasks. If these are not sufficient or compatible, we can use cloud services. For instance for our ML tasks those offered by Azure, Google, etc offer the advantage of easy integration with our data storage. We will locate our data storage and cloud computation on the same service.

For any task that is feasible, we can use our personal machines.

15.5 Tools and Methods to Achieve Requirements

Requirement / Goal	Tools & Technologies	General Methods or Approaches
Inventory Management & Barcode Scanning (P0)	React (frontend), Python + FastAPI/Django (backend), PostgreSQL (database), browser APIs (getUserMedia, Barcode Detector, etc.) or libraries like ZXing JS for scanning items, Material UI for design	RESTful API architecture for CRUD operations. Camera API for barcode scanning. CSV parser for bulk uploads.
Dashboard and Reports (P0)	React components and Chart.js / Tableau, Python (Pandas + NumPy) for aggregation	Query aggregation and scheduled jobs for auto-updates. Summarization and filtering to generate metrics and alerts.
Demand Forecasting (Model Performance Metrics) (P1)	Python (TensorFlow or Scikit-learn), Pandas for data prep	Time-series regression or simple neural networks, and evaluated using Bias and WAPE targets.

Requirement / Goal	Tools & Technologies	General Methods or Approaches
Procurement Advising (P1)	Same backend as forecasting module	Rule-based comparison of stock vs forecasted demand to suggest priority restocks.
Role-Based Access / Authentication (Security & Privacy) (P1)	JWT (JSON Web Tokens)	Role-based access control with token validation.
Audit Logging (P2)	PostgreSQL, Chart.js, MaterialUI	Extended dashboard with donation and fundraiser tracking. PostgreSQL triggers for automatic audit logging of financial and inventory actions.
Load Balancing (P2)	Nginx or HAProxy	Round-robin load distribution algorithm.
Computer Vision (P2)	OpenCV for image processing, and libraries like TensorFlow or Pytorch	Convolutional Neural Network (CNN) for image classification.
Non-Functional Goals (Usability / Performance)	Material UI + CSS modules for consistency, React hooks for state management, and Vite build tool for quick loading times	Conduct internal usability testing with sample users to refine workflows.
Data Storage & Integration	PostgreSQL for core data; cloud Storage for datasets	Versioned data buckets (raw, processed, features); ETL pipelines using Pandas and SQL scripts for reproducibility.

Relation to Performance Metrics and Goals. The selected tools and methods directly support achieving the forecasting performance targets outlined in Section 10.3 (i.e., <30% WAPE and a bias within 10% of actual demand). Python-based machine learning frameworks such as TensorFlow and Scikit-learn provide robust numerical optimization libraries that help minimize forecast error (MAE and WAPE) through model tuning and cross-validation. Pandas and NumPy ensure efficient data preprocessing and normalization, reducing bias by maintaining consistent input distributions. The integration of PostgreSQL and Cloud Storage supports clean, versioned datasets and reproducible training pipelines, which are essential for monitoring model performance over time. Furthermore, scheduled aggregation jobs and dashboard visualizations built with React and Chart.js enable real-time monitoring of prediction accuracy, helping the team iteratively adjust models to meet or surpass the defined performance goals.

16 Proof of Concept Demonstration Plan

We will have a mock (or simple version of our) web application with a limited backend for showing our features. The app will be implemented for admin only (i.e. no role-based access yet) to showcase all the functionality.

The web application will have a skeleton with core features for managing inventory and other administrative tasks. These include but are not limited to: scanning and expiration dates, getting insights into inventory location, status, trends, etc. Logging volunteer schedules/hours.

We will aim to have primitive notification systems and reports. These can in part be generated from the DF algorithm. We do not expect to have the DF algorithm complete for the POC. However we will show meaningful progress and analysis (for example running some early data analysis or toy models). A stretch goal for the POC is to have a skeleton of the architecture linking the DF system to the rest of the app, which at the time of the POC may output Dummy data to showcase this pipeline.

We will also have a clear outline of the UI/UX philosophy of the application, since we are aiming for consistency in this aspect throughout. Our goal is to have some fonts, global styles, colour palettes and format of defined, while the full user interface will not be developed by the POC.

17 Technology Stack

17.1 Programming Languages and Frameworks

The system will be developed using a modern web-based technology stack designed for scalability, maintainability, and ease of collaboration. The core components are as follows:

- **Frontend:** Implemented using `HTML`, `CSS`, and `React.js` with the `Vite` build tool for fast development and hot reloading. The interface will utilize `MaterialUI` to maintain a clean and consistent design system. We will use browser API (e.g. getUserMedia) for camera access and QuaggaJS or ZXingJS library for barcode dectection and decoding.
- **Backend:** Developed using `Python`, depending on integration needs and architecture decision the team will employ `FastAPI` or `Django`. We will deploy a RESTful API. Authentication will be implemented using `JWT` (JSON Web Tokens).
- **Database:** `PostgreSQL` will serve as the primary relational database for reliable data storage and query optimization.
- **Dashboards:** The team will explore the use of `Tableau` or similar visualization libraries to present data insights and inventory metrics.
- **Development Environment:** All development will be conducted using `VS Code`. To maintain consistency across different operating systems, the team will use the `pnpm` package manager and, when needed, `WSL` (Windows Subsystem for Linux) for Windows users. If dependency or configuration issues arise, the team may transition to containerized development using `Docker` to ensure a consistent setup across all members.

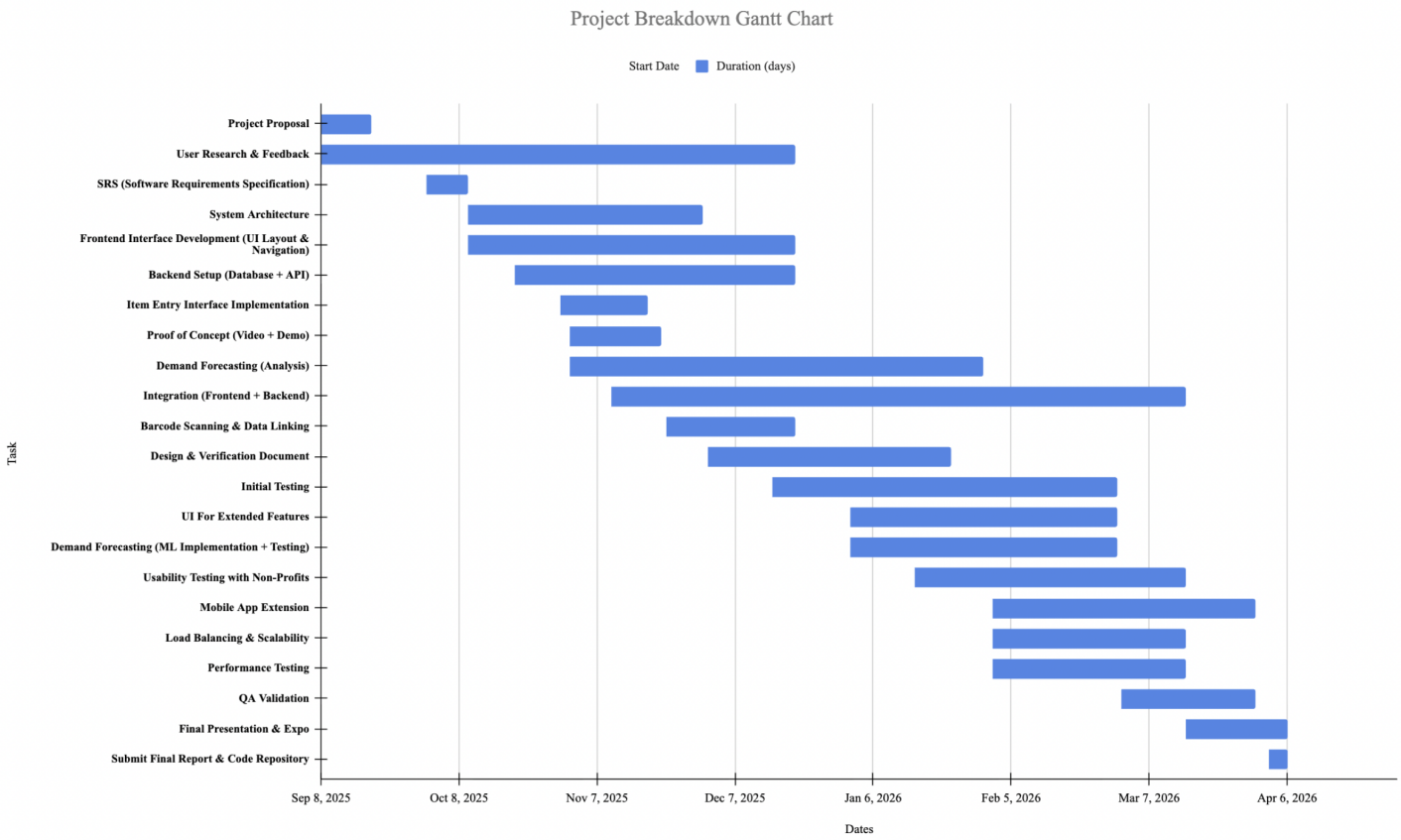
Unit testing will be performed using the `unittest` framework to verify functionality.

17.2 ML Libraries (If Applicable)

If ML components such as DF or object recognition are implemented, the following libraries will be used:

- `NumPy` and `Pandas` for data preprocessing and numerical computation.
- `TensorFlow` for model training and evaluation.
- `OpenCV` for computer-vision tasks, such as identifying non-barcoded items.

17.3 Hardware and Other Technological Considerations

The project does not require GPU acceleration; all training and computation can be performed on CPU resources. Source code will be version-controlled using `Git` and hosted on `GitHub` to support collaboration, code reviews, and continuous integration throughout the development lifecycle.

18 Project Schedule: Gantt Chart